



Code Cultivation

Object-Oriented Garden Systems

Summary: Build a comprehensive digital garden ecosystem while discovering advanced Python concepts. Create tools to manage community gardens efficiently through data-driven approaches.

Version: 3.0

Contents

I	Foreword	2
II	AI Instructions	3
III	Introduction	5
IV	General Instructions	6
V	Exercise 0: Planting Your First Seed	7
VI	Exercise 1: Garden Data Organizer	9
VII	Exercise 2: Plant Growth Simulator	11
VIII	Exercise 3: Plant Factory	13
IX	Exercise 4: Garden Security System	15
X	Exercise 5: Specialized Plant Types	17
XI	Exercise 6: Garden Analytics	19
XII	Turn in and Submission	21

Chapter I

Foreword

In the digital age, even gardens need smart systems.

A wise gardener once said: "You can't plant flowers and then pull them up every day to see how the roots are doing." The same applies to code—sometimes the most beautiful growth happens when we trust the process and let our programs develop naturally, layer by layer.

Behind every thriving community garden lies a network of relationships. Plants don't grow in isolation; they form communities where each species contributes something unique while sharing common needs. Some plants protect others from pests, some fix nitrogen in the soil, and others provide shade for delicate seedlings. This interconnected web of mutual support mirrors how well-designed software systems work—individual components with distinct roles, working together harmoniously.

Just as a master gardener knows that healthy soil produces healthy plants, experienced programmers understand that well-structured code grows into robust applications. You'll discover that organizing your code is like planning a garden: some elements need protection, others need room to grow, and the best systems emerge when everything has its proper place and purpose.

In this project, you'll cultivate your programming skills while building tools that could genuinely help community gardens flourish. Every line of code you write is a seed planted in the digital soil of possibility.

Chapter II

AI Instructions

● Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

● Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

● Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

● Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

● Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter III

Introduction

Welcome to CodeCultivation!

Building on your Python fundamentals from the first activity, you'll now tackle more complex programming challenges by creating a comprehensive garden data management system. This project introduces advanced concepts that make Python a powerful tool for modeling real-world systems.

You'll work on:

- Understanding how Python programs are structured and executed
- Organizing data using an object-oriented approach
- Creating reusable code components
- Building systems that can adapt and extend
- Protecting data integrity in collaborative environments
- Designing scalable software architectures

Each exercise builds on the previous ones, creating a complete digital garden ecosystem by the end.



IMPORTANT: This module starts with basic Python program structure, then progresses to Object-Oriented Programming. Each exercise should contain the requested definitions and any required code. You may include simple test code at the bottom of each file using `if __name__ == "__main__":` blocks for your own testing.

Chapter IV

General Instructions

- Your code must be written in Python 3.10+
- Your code must respect the `flake8` linter standards
- Each exercise must be in its own file and directory
- Use proper naming conventions: classes in PascalCase, functions and variables in snake_case
- All functions and methods must include type hints: use `mypy` to check your code
- You don't need to handle input validation unless explicitly mentioned
- Focus on demonstrating programming concepts clearly
- Your programs must always run without errors



Digital Garden Ecosystem Note: This project focuses on Python programming concepts, starting from basic program structure and progressing to Object-Oriented Programming. Each exercise introduces new features while building a cohesive garden management system. Later exercises will reuse concepts from earlier ones.



Object-Oriented Programming: Starting from Exercise 1, all exercises in this module require the use of classes. Python keywords such as `'class'` and `'def'` are fundamental language keywords and do not need to be listed in the authorized functions for each exercise.



For a nicer display, you can use the `capitalize()` method of an `str` (string) object. This will not be part of the "Authorized functions", but you can still use it if you want.

Chapter V

Exercise 0: Planting Your First Seed

	Exercise0
ft_garden_intro	
Directory: <i>ex0/</i>	
Files to Submit: <code>ft_garden_intro.py</code>	
Authorized: <code>print()</code>	

Before we can build complex garden management systems, you need to understand how Python programs work.

Every Python program needs a starting point - a place where execution begins. It's just like planting your first seed in a garden!

Your task is to create your very first Python program that displays information about a plant in your garden.

Requirements:

- Create a program that runs when executed directly using `python3 ft_garden_intro.py`
- Use the special `if __name__ == "__main__":` pattern.
- Store the following plant information in simple variables: name, height, and age.
- Display the plant information using `print()`
- Mimic the output display below; there will not be a strict output check during the evaluation.

Example:

```
$> python3 ft_garden_intro.py
=== Welcome to My Garden ===
Plant: Rose
Height: 25cm
Age: 30 days

=== End of Program ===
```



Understanding how programs start and execute is fundamental before moving to more complex concepts such as classes and other object-oriented features.

Note: Starting from this exercise and for all following exercises in this module (and future Python projects), you are allowed to use `if __name__ == "__main__":` blocks for testing your code. This pattern will not be explicitly displayed in the authorized functions, but it remains available during the entire project.



You must be able to explain why using the `if __name__ == "__main__":` line is important. This will lead you to the concept of imports, which will be part of an upcoming project. It is, however, important to introduce this good practice now.



Have you noticed that some Python files start with a special line beginning with `#!`? Research what this "shebang" line does and why it might be useful for making your scripts executable directly. The evaluation will ask you to update your script live to use this "shebang".

Chapter VI

Exercise 1: Garden Data Organizer

	Exercise1
ft_garden_data	
Directory: <i>ex1/</i>	
Files to Submit: <code>ft_garden_data.py</code>	
Authorized: <code>print()</code>	

The community garden needs to track multiple plants with their information. You need to store and display data for several plants efficiently.

Each plant has:

- A name
- Height in centimeters
- Age in days

Create a program that manages data for at least 3 different plants and displays their information in an organized way.

Required: You must create a `Plant` *class* that serves as a model for any plant, rather than handling each one individually. All plants will be described using the same *attributes*, or characteristics, listed above: name, height and age. For each plant, the class will be *instantiated*, then the attributes will be set to their specific values.

You will create a `show()` *method* in the class to display the plant information.

Example:

```
$> python3 ft_garden_data.py
=== Garden Plant Registry ===
Rose: 25cm, 30 days old
Sunflower: 80cm, 45 days old
Cactus: 15cm, 120 days old
```



How are you currently storing plant information? There are multiple ways to do this. What challenges might arise with more plants? In the next exercises, you may eventually need to make your approach evolve.



If you have already worked with Python programming or object-oriented programming in the past, you will probably go directly for an advanced solution. In this case, you may find the next exercises a little bit redundant.